WePayU

Sistema de Gerenciamento de Folha de Pagamento

Documentação Técnica Abrangente da Arquitetura, Classes, Métodos e Regras de Negócio

Cobertura Integral das User Stories 1 a 10 • Arquitetura Refatorada

Disciplina: Programação 2 (P2)

Instituição: Universidade Federal de Alagoas (UFAL)

Linguagem: Java (JDK 25 / compatível com Java 17+)

Padrão Arquitetural: Orientação a Objetos e Padrões GoF

Testes de Aceitação: EasyAccept (100% de Aprovação em 786 Testes)

Data de Emissão: 2026

Sumário

1. Visão Geral do Sistema — Objetivos, funcionalidades e categorias de empregados

2. Arquitetura de Software e Padrões de Projeto — Separação em pacotes, Façade, Memento e Polimorfismo Dinâmico

3. Resumo das User Stories Implementadas (US1 a US10) — Visão funcional detalhada

4. Documentação das Classes e Métodos

4.1. Camada de Fachada: `br.ufal.ic.p2.wepayu.Facade`

4.2. Núcleo do Sistema: `br.ufal.ic.p2.wepayu.core.Sistema`

4.3. Motor de Cálculo Financeiro: `br.ufal.ic.p2.wepayu.core.FolhaDePagamento`

4.4. Modelo de Domínio: `Empregado`, `EmpregadoHorista`, `EmpregadoAssalariado`, `EmpregadoComissionado`

4.5. Sistema Polimórfico de Agendas: `AgendaPagamento`, `AgendaSemanal`, `AgendaMensal`

4.6. Classes Especializadas: `MetodoPagamento`, `FiliacaoSindical`, `Sindicato`, `FabricaEmpregados`

4.7. Registros Auxiliares: `CartaoPonto`, `ResultadoVenda`, `TaxaServico`, `DadosPagamento`

4.8. Pacote de Exceções Personalizadas: `br.ufal.ic.p2.wepayu.exceptions`

5. Ciclo de Vida de Execução, Undo/Redo e Persistência

6. Guia de Compilação e Execução dos Testes

1. Visão Geral do Sistema

O **WePayU** é um sistema corporativo de gestão e processamento da folha de pagamento de colaboradores de uma organização. O software provê um conjunto completo de operações para cadastro de empregados, registro de jornadas diárias, apuração de comissões por vendas, controle de mensalidades e serviços sindicais, emissão detalhada de contracheques periódicos, suporte a agendas de pagamento flexíveis e customizadas, além de um histórico transacional completo que permite operações de **Undo** (desfazer) e **Redo** (refazer).

1.1. Categorias de Empregados Atendidas

- **Horistas (`EmpregadoHorista`):** Recebem remuneração calculada estritamente pelas horas trabalhadas, registradas em cartões de ponto diários (`CartaoPonto`). A jornada padrão diária é de até 8 horas normais; horas trabalhadas além desse limite são bonificadas com taxa adicional de 50% (fator 1.5x). Por padrão, a folha é apurada na agenda `"semanal 5"` (toda sexta-feira), mas o colaborador pode adotar qualquer agenda disponível. Salário horário armazenado numericamente como `double`.
- **Assalariados (`EmpregadoAssalariado`):** Recebem um valor salarial mensal fixo contratual (armazenado como `double`). Por padrão, recebem na agenda `"mensal $"` (último dia útil de cada mês). Caso alocado em agenda semanal (ex.: `"semanal 5"`), o salário é calculado proporcionalmente às 52 semanas do ano: `(salario × 12 / 52) × semanas`.
- **Comissionados (`EmpregadoComissionado`):** Especialização de empregado assalariado que recebe um salário fixo somado a uma comissão percentual aplicada sobre o valor bruto das vendas comprovadas. Salário e taxa de comissão são armazenados numericamente como `double`. Por padrão, recebem na agenda `"semanal 2 5"` (a cada duas semanas, na sexta-feira).

1.2. Métodos de Pagamento com Classes Próprias

O sistema adota uma hierarquia polimórfica sob a interface `MetodoPagamento`:

- `MetodoPagamentoEmMaos`: Pagamento em dinheiro/cheque entregue em mãos (padrão do sistema).
- `MetodoPagamentoCorreios`: Envio de cheque nominal via correio para o endereço residencial cadastrado.
- `MetodoPagamentoBanco`: Depósito bancário direto, encapsulando nome do banco, agência e conta corrente.

1.3. Gestão Sindical e Deduções

Toda a complexidade de afiliação sindical é encapsulada na classe própria `FiliacaoSindical`. Ela armazena o status do colaborador, sua identificação sindical única, o valor da taxa diária, o histórico de taxas de serviços adicionais e o saldo devedor acumulado.

No fechamento da folha, todas as taxas e débitos sindicais são deduzidos do salário bruto. Se as deduções superarem o salário disponível, o saldo excedente é acumulado como débito pendente para quitação nas folhas seguintes.

2. Arquitetura de Software e Padrões de Projeto

2.1. Organização em Pacotes Especializados

A arquitetura foi modularizada em quatro pacotes com responsabilidades estritamente delimitadas:

- `br.ufal.ic.p2.wepayu`: Contém a `Facade`, mantendo a compatibilidade estrita com os testes de aceitação do EasyAccept.
- `br.ufal.ic.p2.wepayu.core`: Concentra a inteligência do sistema, abrigando `Sistema` (gestor de estado e transações), `FolhaDePagamento` (motor financeiro e relatórios), `ValidadorEmpregado` e `ValidadorAgenda`.
- `br.ufal.ic.p2.wepayu.models`: Entidades ricas de domínio (`Empregado`, especializações, hierarquia de `AgendaPagamento`, `MetodoPagamento`, `FiliacaoSindical` e registros de cartões/vendas/taxas/dados da folha).
- `br.ufal.ic.p2.wepayu.exceptions`: Conjunto completo de 45 exceções personalizadas para cada caso de erro do sistema (em pacote estritamente minúsculo).

2.2. Polimorfismo Dinâmico e Despacho de Métodos

A distinção de comportamento entre as diferentes categorias de empregados e rotinas de pagamento é estruturada por meio de **despacho dinâmico de métodos** e polimorfismo de sobrescrita:

- O cálculo de contracheques é delegado para `empregado.processarFolha(data, commit)`, onde cada subclasse implementa sua fórmula específica e formata sua linha do relatório.
- O cálculo da base salarial periódica é delegado para `agenda.calcularSalarioBase(salarioMensal)`.
- A verificação de dia de pagamento é delegada para `agenda.ehDiaDePagamento(data)`.
- Operações específicas de horistas (como `adicionarCartao`, `getHorasNormaisTrabalhadas`) lançam `EmpregadoNaoEhHoristaException` na superclasse e são sobrescritas em `EmpregadoHorista`.
- Operações específicas de comissionados (como `adicionarVenda`, `getVendasRealizadas`) lançam `EmpregadoNaoEhComissionadoException` na superclasse e são sobrescritas em `EmpregadoComissionado`.

2.3. Padrão Façade (Fachada Pura GoF)

A classe `Facade` possui menos de 105 linhas e atua como puro delegador GoF. Ela não executa validação, não instancia regras de negócio e não manipula coleções; repassa 100% das chamadas diretamente para o `Sistema`.

2.4. Padrão Memento com Snapshots em Memória (Undo / Redo)

Para viabilizar a reversão segura de qualquer comando (US8), a classe `Sistema` intercepta todas as operações modificadoras através do executor transacional `executarComandoModificador`:

- Antes da modificação, serializa um snapshot do objeto `EstadoSistema` em array de bytes em memória (`byte[]`), englobando empregados, próximo ID, folhas geradas e agendas disponíveis.
- Executa o comando. Se concluído com êxito, empilha o snapshot anterior na `undoStack` e esvazia a `redoStack` .
- Caso ocorra qualquer erro de validação (exceção), o estado anterior é restaurado imediatamente (**Rollback**), mantendo o sistema íntegro e sem poluir o histórico.

3. Resumo das User Stories Implementadas

História	Título	Descrição e Responsabilidade Principal
US1 / US1_1	Criação de Empregado	Inclusão de empregados horistas, assalariados e comissionados. Geração de ID único, validações de campos obrigatórios e formato numérico. Persistência em disco validada em US1_1.
US2 / US2_1	Remoção de Empregado	Exclusão de colaboradores cadastrados via ID. Validação de existência do ID informado.
US3 / US3_1	Cartão de Ponto	Lançamento de horas diárias trabalhadas para horistas via classe `CartaoPonto`. Consulta de horas normais (até 8h) e horas extras em intervalos de datas [dataInicial, dataFinal).
US4 / US4_1	Lançamento de Venda	Registro de comprovantes de vendas (`ResultadoVenda`) para comissionados. Consulta de vendas efetuadas em período de datas.
US5 / US5_1	Taxas de Serviço Sindicais	Lançamento de débitos de serviços prestados pelo sindicato via ID de membro sindical (`TaxaServico`). Consulta agregada de taxas no período.
US6 / US6_1	Alteração de Empregado	Atualização cadastral completa: nome, endereço, troca dinâmica de tipo de contrato com preservação de histórico, dados sindicais, dados bancários e agenda de pagamento.
US7	Rodar Folha de Pagamento	Cálculo financeiro consolidado dos pagamentos devidos na data informada, apuração de horas, comissões, deduções de taxas sindicais, débitos acumulados e geração de relatório tabular formatado idêntico às especificações oficiais.
US8	Undo / Redo	Capacidade de desfazer e refazer qualquer comando modificador executado desde o início da sessão. Isolamento transacional com rollback automático diante de erros.
US9 / US9_1	Agendas de Pagamentos	Atributo `agendaPagamento` atribuído aos empregados. Default por tipo ("semanal 5", "mensal $", "semanal 2 5"). Mudança de agenda e cálculo financeiro conforme periodicidade individual. Persistência validada em US9_1.

História	Título	Descrição e Responsabilidade Principal
US10 / US10_1	Criação de Agendas Customizadas	Comando `criarAgendaDePagamentos(descricao)` para criação de novas agendas semanais (`semanal [N] D`) e mensais (`mensal D|$`). Validações rigorosas de limites e duplicação. Persistência validada em US10_1.

4. Documentação das Classes e Métodos

4.1. Classe `Facade`

CAMADA DE FACHADA `br.ufal.ic.p2.wepayu.Facade`

Ponto de integração com os scripts do EasyAccept. Instancia a classe `Sistema` recuperando o arquivo `sistema.dat` e repassa as operações.

`public void zerarSistema() throws Exception`

Limpa todos os cadastros, restaura agendas padrão e reseta dados de folhas geradas. Reversível via `undo()`.

`public String criarEmpregado(String nome, String endereco, String tipo, String salario) throws Exception`

Cria um novo empregado horista ou assalariado. Retorna o ID gerado.

`public String criarEmpregado(String nome, String endereco, String tipo, String salario, String comissao) throws Exception`

Cria um empregado do tipo comissionado com salário mensal e comissão percentual sobre vendas. Retorna o ID gerado.

`public String getAtributoEmpregado(String emp, String atributo) throws Exception`

Recupera qualquer atributo do empregado: nome, endereco, tipo, salario, comissao, sindicalizado, idSindicato, taxaSindical, metodoPagamento, banco, agencia, contaCorrente, agendaPagamento.

`public void alteraEmpregado(String emp, String atributo, String valor) throws Exception`

Altera atributos univalorados (incluindo agendaPagamento com verificação de disponibilidade).

`public void criarAgendaDePagamentos(String descricao) throws Exception`

Registra uma nova agenda de pagamentos semanal ou mensal customizada no sistema.

`public String totalFolha(String data) throws Exception`

Retorna o valor total bruto da folha de pagamento na data especificada, sem persistir alterações.

`public void rodaFolha(String data, String saida) throws Exception`

Calcula a folha, gera o arquivo de texto formatado em ISO-8859-1 e, somente após a escrita bem-sucedida no disco, comita as alterações cadastrais (débitos sindicais).

`public void undo() throws Exception / public void redo() throws Exception`

Desfaz ou refaz comandos modificadores através de pilha de snapshots em memória.

`public void encerrarSistema() throws Exception`

Bloqueia comandos adicionais e persiste o estado consolidado no arquivo `sistema.dat`.

4.2. Classe `Sistema`

CONTROLADOR CENTRAL `br.ufal.ic.p2.wepayu.core.Sistema`

Responsável pela integridade e persistência das coleções de empregados, agendas disponíveis e controle transacional de Undo/Redo. Possui verificação de encerramento (`checarSistemaAberto()`) em todos os métodos públicos.

4.3. Classe `FolhaDePagamento`

PROCESSAMENTO FINANCEIRO `br.ufal.ic.p2.wepayu.core.FolhaDePagamento`

Responsável pela montagem e consolidação do relatório financeiro. O processamento individual de cada colaborador é delegado diretamente para `empregado.processarFolha(data, commit)`, consolidando os objetos `DadosPagamento` retornados em seções ordenadas:

- `processarHoristas(...)`: Consolida colaboradores horistas cujo dia de pagamento coincide com a data informada.
- `processarAssalariados(...)`: Consolida colaboradores assalariados com vencimento na data.
- `processarComissionados(...)`: Consolida comissionados com vencimento na data, agregando comissões e salário fixo.

4.4. Modelo de Domínio (Entidades de Empregado)

MODELOS Pacote `br.ufal.ic.p2.wepayu.models`

`Empregado.java` (Classe Abstrata)

Superclasse de todos os colaboradores. Encapsula `AgendaPagamento`, `FiliacaoSindical` e `MetodoPagamento`. Declara o método abstrato `processarFolha(LocalDate, boolean)` que permite ao motor da folha executar cálculos polimórficos sem conhecer detalhes internos de cada categoria.

`EmpregadoHorista.java`

Inicializa com a agenda default `"semanal 5"`. Em `processarFolha`, verifica se a agenda estipula pagamento na data, busca as horas no intervalo retornado por `agenda.getInicioPeriodo(data)`, aplica as horas extras a 1.5x e desconta taxas sindicais acumuladas.

`EmpregadoAssalariado.java`

Inicializa com a agenda default `"mensal $"`. Em `processarFolha`, utiliza `agenda.calcularSalarioBase(salario)` para apurar a remuneração devida no período (mensal integral ou semanal proporcional a 52 semanas).

`EmpregadoComissionado.java`

Inicializa com a agenda default `"semanal 2 5"`. Em `processarFolha`, calcula o fixo via `agenda.calcularSalarioBase(salario)` somado às vendas no período multiplicadas pela taxa de comissão.

4.5. Sistema Polimórfico de Agendas de Pagamento

AGENDAS Implementações de `AgendaPagamento`

> **`public interface AgendaPagamento extends Serializable`**
> Define os contratos essenciais: `getDescricao()`, `ehDiaDePagamento(LocalDate)`, `getInicioPeriodo(LocalDate)`, `getDiasTaxaSindical(LocalDate)` e `calcularSalarioBase(double)`.

`AgendaSemanal.java`

Modela agendas semanais regulares (`semanal D`) ou periódicas a cada N semanas (`semanal N D`), onde N varia de 1 a 52 e D de 1 (segunda-feira) a 7 (domingo). Calcula ocorrências exatas no ano e apura a base salarial com `Math.floor((salario × 12 × semanas / 52) × 100) / 100`.

`AgendaMensal.java`

Modela agendas mensais em dia fixo (1 a 28) ou no último dia útil do mês (`mensal $`). A regra do último dia útil retrocede automaticamente finais de semana (sábado ou domingo), garantindo cumprimento de todas as diretrizes de legislação e testes do professor.

`ValidadorAgenda.java`

Valida e converte strings de descrição em instâncias de `AgendaSemanal` ou `AgendaMensal`, validando limites numéricos e lançando `DescricaoDeAgendaInvalidaException` diante de especificações incorretas.

4.6. Classes Especializadas e Registros Auxiliares

- **`FabricaEmpregados.java`**: Criação segura de instâncias e conversão de tipos (`converterTipo`) com transferência de histórico.
- **`Sindicato.java`**: Serviço de validação de identificações sindicais únicas e gerenciamento de taxas de serviço.
- **`MetodoPagamento.java`**: Hierarquia para `MetodoPagamentoEmMaos`, `MetodoPagamentoCorreios` e `MetodoPagamentoBanco`.
- **`FiliacaoSindical.java`**: Encapsulamento de vínculo sindical, saldo devedor e lista de taxas.
- **`CartaoPonto.java`**: Armazena data da jornada e total de horas (`double`), provendo `getHorasNormais()` e `getHorasExtras()`.
- **`ResultadoVenda.java`**: Armazena data e valor monetário da venda (`double`).
- **`TaxaServico.java`**: Armazena data e valor monetário da taxa sindical avulsa (`double`).
- **`DadosPagamento.java`**: Objeto imutável com todos os 9 atributos `private final` que encapsula os resultados do cálculo de um empregado na folha.

4.7. Pacote de Exceções Personalizadas

EXCEÇÕES Pacote `br.ufal.ic.p2.wepayu.exceptions`

Todas as 45 exceções do sistema são classes independentes que estendem `WePayUException`. Cada classe define a mensagem literal exigida pelas especificações dos testes de aceitação:

Classe de Exceção	Mensagem Literal
`IdentificacaoEmpregadoNulaException`	"Identificacao do empregado nao pode ser nula."
`EmpregadoNaoExisteException`	"Empregado nao existe."
`NomeNaoPodeSerNuloException`	"Nome nao pode ser nulo."
`EnderecoNaoPodeSerNuloException`	"Endereco nao pode ser nulo."
`SalarioNaoPodeSerNuloException`	"Salario nao pode ser nulo."
`SalarioDeveSerNumericoException`	"Salario deve ser numerico."
`SalarioDeveSerNaoNegativoException`	"Salario deve ser nao-negativo."
`ComissaoNaoPodeSerNulaException`	"Comissao nao pode ser nula."
`ComissaoDeveSerNumericaException`	"Comissao deve ser numerica."
`ComissaoDeveSerNaoNegativaException`	"Comissao deve ser nao-negativa."
`TipoInvalidoException`	"Tipo invalido."
`TipoNaoAplicavelException`	"Tipo nao aplicavel."
`AtributoNaoExisteException`	"Atributo nao existe."
`EmpregadoNaoEhHoristaException`	"Empregado nao eh horista."
`EmpregadoNaoEhComissionadoException`	"Empregado nao eh comissionado."
`EmpregadoNaoEhSindicalizadoException`	"Empregado nao eh sindicalizado."
`EmpregadoNaoRecebeEmBancoException`	"Empregado nao recebe em banco."
`DataInvalidaException`	"Data invalida."
`DataInicialInvalidaException`	"Data inicial invalida."
`DataFinalInvalidaException`	"Data final invalida."

Classe de Exceção	Mensagem Literal
`DataInicialNaoPodeSerPosteriorException`	"Data inicial nao pode ser posterior aa data final."
`HorasDevemSerNumericasException`	"Horas devem ser numericas."
`HorasDevemSerPositivasException`	"Horas devem ser positivas."
`ValorDeveSerNumericoException`	"Valor deve ser numerico."
`ValorDeveSerPositivoException`	"Valor deve ser positivo."
`MetodoPagamentoInvalidoException`	"Metodo de pagamento invalido."
`ValorDeveSerTrueOuFalseException`	"Valor deve ser true ou false."
`IdentificacaoSindicatoNulaException`	"Identificacao do sindicato nao pode ser nula."
`IdentificacaoSindicatoDuplicadaException`	"Ha outro empregado com esta identificacao de sindicato"
`TaxaSindicalNaoPodeSerNulaException`	"Taxa sindical nao pode ser nula."
`TaxaSindicalDeveSerNumericaException`	"Taxa sindical deve ser numerica."
`TaxaSindicalDeveSerNaoNegativaException`	"Taxa sindical deve ser nao-negativa."
`BancoNaoPodeSerNuloException`	"Banco nao pode ser nulo."
`AgenciaNaoPodeSerNulaException`	"Agencia nao pode ser nulo."
`ContaCorrenteNaoPodeSerNulaException`	"Conta corrente nao pode ser nulo."
`IdentificacaoMembroNulaException`	"Identificacao do membro nao pode ser nula."
`MembroNaoExisteException`	"Membro nao existe."
`NaoHaEmpregadoComEsseNomeException`	"Nao ha empregado com esse nome."
`NaoHaComandoADesfazerException`	"Nao ha comando a desfazer."
`NaoHaComandoARefazerException`	"Nao ha comando a refazer."
`SistemaEncerradoException`	"Nao pode dar comandos depois de encerrarSistema."
`AgendaNaoDisponivelException`	"Agenda de pagamento nao esta disponivel"
`DescricaoDeAgendaInvalidaException`	"Descricao de agenda invalida"
`AgendaJaExisteException`	"Agenda de pagamentos ja existe"

5. Ciclo de Vida de Execução, Undo/Redo e Persistência

O ciclo de vida do WePayU em uma sessão padrão ocorre conforme o fluxo abaixo:

1. **Inicialização:** Ao instanciar `Facade`, o método `Sistema.carregar()` recupera o arquivo `sistema.dat`. Caso não exista, instancia um novo `Sistema` com coleções vazias e agendas padrão (`"semanal 5"`, `"semanal 2 5"`, `"mensal $"`).
2. **Execução de Comandos:** A cada comando modificador, um snapshot do estado em memória é empilhado na `undoStack` e a `redoStack` é limpa. Se houver falha de validação, ocorre rollback automático.
3. **Consultas:** Consultas a atributos, horas, vendas, taxas e totalização da folha não afetam as pilhas de histórico.
4. **Desfazer / Refazer:** Comandos `undo()` e `redo()` transferem os snapshots entre as pilhas e substituem os dados em memória sem perda de integridade.
5. **Encerramento:** O comando `encerrarSistema()` bloqueia a aceitação de novos comandos e persiste os dados consolidados no arquivo binário `sistema.dat`.

6. Guia de Compilação e Execução dos Testes

6.1. Compilação do Código Fonte

No terminal PowerShell na raiz do projeto:

```
javac -parameters -cp "lib/easyaccept.jar;src" -d out (Get-ChildItem -Recurse -Filter *.java src | Fo
```

6.2. Execução da Suíte de Testes EasyAccept

```
java -cp "out;lib/easyaccept.jar" Main
```

> **Resultado Verificado:** Todos os 786 testes formais do EasyAccept das User Stories US1 a US10_1 executam com **100% de sucesso (0 erros, 0 falhas)**.